Universidade do Minho
Escola de Engenharia

Departamento de Informática

Licenciatura em Engenharia Informática

Computação Gráfica

Trabalho Prático (Fase 3)

Grupo n.º 28

Carlos Daniel Lopes Cunha, nº A106910

Pedro Nuno de Bastos Pinho Costa, nº A107375

Francisco Queirós Ribeiro da Costa Maia, nº A108962

Índice

1 Introdução **1**

2 Modelação de Superfícies de Bézier **2**
2.1 Leitura e Estruturação de Dados 2
2.2 Cálculo Analítico da Superfície 2
2.3 Tesselação e Geração da Malha 2

3 Curvas de Catmull-Rom e Animações **3**
3.1 Implementação de Curvas de Catmull-Rom 3
3.2 Alinhamento e Orientação (*Align*) 4
3.3 Controlo Temporal de Rotações 4

4 Vertex Buffer Objects (VBOs) **4**
4.1 Transição do Paradigma de Desenho 4
4.2 Renderização de Alto Desempenho 5

5 Sistema Solar Dinâmico **6**
5.1 Hierarquia e Movimentos Orbitais 6
5.2 O Cometa e as Superfícies de Bézier 6

6 Conclusão **8**

1 Introdução

O presente relatório detalha o desenvolvimento da terceira fase do projeto prático da Unidade Curricular de Computação Gráfica.

Na vertente da modelação (*Generator*), o principal objetivo desta fase consistiu na expansão do leque de primitivas gráficas, adicionando capacidade de interpretar ficheiros de pontos de controlo para gerar modelos baseados em *patches* de Bézier, com níveis de tesselação dinâmicos.

Relativamente ao motor gráfico (*Engine*), as atualizações incidiram sobre duas áreas fundamentais. A primeira aborda a introdução de dinâmicas na cena, abandonando o paradigma estático. Para tal, implementaram-se transformações baseadas no tempo, destacando-se as rotações contínuas e as translações ao longo de curvas cúbicas de Catmull-Rom.

A segunda grande alteração no motor gráfico foca-se na performance. Procedeu-se à reestruturação completa do processo de desenho da geometria, substituindo *Immediate Mode* por *Vertex Buffer Objects* (VBOs). Esta transição permitiu delegar o armazenamento dos vértices para a memória da placa gráfica (VRAM), melhorando substancialmente a taxa de atualização (*framerate*) das cenas renderizadas.

Ao longo deste relatório, serão expostas as decisões metodológicas e arquiteturais tomadas durante a implementação destas novas funcionalidades.

2 Modelação de Superfícies de Bézier

O processo de geração da superfície foi dividido em três etapas:

2.1 Leitura e Estruturação de Dados

O ficheiro de entrada fornecido à aplicação contém a definição geométrica da superfície. A leitura inicia-se pela extração do número de *patches*, seguido dos índices que compõem cada um. Como estamos perante superfícies bicúbicas, cada *patch* é estritamente definido por uma grelha de 4x4 pontos, totalizando 16 índices por *patch*. Estes dados foram armazenados numa estrutura auxiliar (`struct Patch`). Seguidamente, o algoritmo procede à leitura das coordenadas tridimensionais de todos os pontos de controlo disponíveis, armazenando-os num vetor.

2.2 Cálculo Analítico da Superfície

Para determinar um ponto exato na superfície tridimensional, recorreu-se à equação geral de um *patch* de Bézier, que depende de dois parâmetros u e v, ambos contidos no intervalo $[0, 1]$. A posição de um ponto $P(u, v)$ é dada pelo somatório ponderado dos 16 pontos de controlo do *patch* ($P_{i,j}$), utilizando os polinómios de Bernstein de grau 3 (B^3) como funções de base:

$$P(u, v) = \sum_{i=0}^{3} \sum_{j=0}^{3} B_i^3(u) \cdot B_j^3(v) \cdot P_{i,j} \tag{1}$$

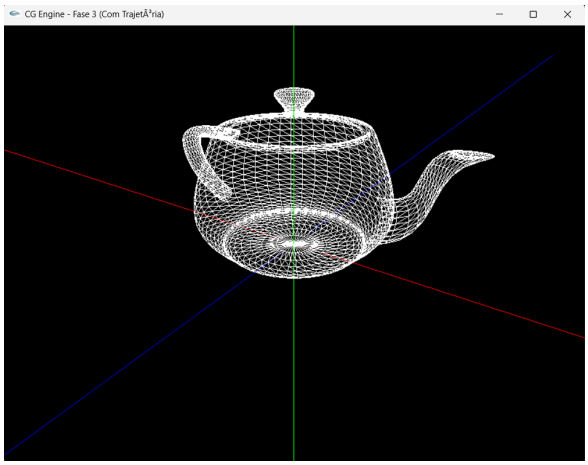
Na implementação prática (através da função `getBezierPoint`), os polinómios de Bernstein foram traduzidos diretamente para código, calculando os vetores de pesos `bu` e `bv` para as componentes u e v, respetivamente.

2.3 Tesselação e Geração da Malha

Sendo o OpenGL otimizado para a renderização de polígonos, a superfície paramétrica contínua tem de ser aproximada por uma malha de triângulos. O grau de detalhe desta malha é controlado pelo nível de tesselação (*tessellation*), passado como argumento na linha de comandos.

O algoritmo calcula o *step* dividindo 1.0 pelo nível de tesselação. Através de um duplo ciclo iterativo, os parâmetros u e v avançam gradualmente ao longo do *patch*. Em cada iteração de sub-quadrícula delimitada pelos vértices correspondentes a (u, v), $(u+step, v)$, $(u + step, v + step)$ e $(u, v + step)$, são gerados dois triângulos (6 vértices) garantindo a orientação correta das faces (sentido oposto aos ponteiros do relógio).

O conjunto final de vértices processados para todos os *patches* é por fim exportado para o ficheiro `.3d`, preservando o formato adotado nas fases anteriores, estando assim pronto a ser consumido pelo motor gráfico.



Teapot gerado

3 Curvas de Catmull-Rom e Animações

Nesta fase, o motor gráfico evoluiu de uma visualização estática para um sistema dinâmico, onde o movimento é controlado pelo tempo e por trajetórias curvilíneas complexas.

3.1 Implementação de Curvas de Catmull-Rom

Para a translação de objetos ao longo de caminhos curvilíneos, recorreu-se às *splines* cúbicas de Catmull-Rom. Ao contrário de outras curvas paramétricas, estas garantem a passagem por todos os pontos de controlo definidos no ficheiro XML. Na implementação adotada, os pontos formam um ciclo fechado: o ponto "anterior" ao primeiro e o ponto "seguinte" ao último são obtidos por aritmética modular sobre o vetor de controlo, eliminando a necessidade de pontos auxiliares explícitos e garantindo automaticamente a continuidade e a ciclicidade da animação. O número mínimo de pontos exigido é quatro.

O motor calcula a posição de um objeto no instante t através da interpolação entre quatro pontos de controlo consecutivos (P_{i-1}, P_i, P_{i+1}, P_{i+2}). A implementação utiliza a matriz característica de Catmull-Rom para determinar tanto a posição como a tangente em cada instante:

$$P(u) = \begin{bmatrix} u^3 & u^2 & u & 1 \end{bmatrix} \begin{bmatrix} -0.5 & 1.5 & -1.5 & 0.5 \\ 1.0 & -2.5 & 2.0 & -0.5 \\ -0.5 & 0.0 & 0.5 & 0.0 \\ 0.0 & 1.0 & 0.0 & 0.0 \end{bmatrix} \begin{bmatrix} P_{i-1} \\ P_i \\ P_{i+1} \\ P_{i+2} \end{bmatrix} \tag{2}$$

A função `getGlobalCatmullRomPoint` processa o tempo global da cena, mapeando-o para o intervalo $[0, 1]$ da curva e selecionando os índices dos pontos de controlo adequados através de módulo aritmético, garantindo assim animações sem descontinuidades.

3.2 Alinhamento e Orientação (*Align*)

Sempre que a *flag* `align` se encontra ativa, o motor calcula a orientação do objeto de forma a que este "olhe" na direção do movimento. O processo é realizado na função `buildRotationMatrix` e segue os passos seguintes:

1. A derivada da curva no instante atual é normalizada, obtendo-se o eixo local de avanço Z_{local}.
2. Usando um vetor *Up* auxiliar fixo $(0, 1, 0)$, calcula-se o eixo lateral: $X_{\text{local}} = \text{normalize}(Up \times Z_{\text{local}})$.
3. O eixo vertical local é então obtido por: $Y_{\text{local}} = \text{normalize}(Z_{\text{local}} \times X_{\text{local}})$.

Os três vetores ortonormados formam as colunas da matriz de rotação 4×4 que é passada a `glMultMatrixf`. Como o OpenGL utiliza convenção *column-major*, a matriz é construída de forma a que as componentes de X_{local}, Y_{local} e Z_{local} ocupem, respetivamente, as colunas 0, 1 e 2 da representação linear:

$$M = \begin{bmatrix} X_x & Y_x & Z_x & 0 \\ X_y & Y_y & Z_y & 0 \\ X_z & Y_z & Z_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{3}$$

3.3 Controlo Temporal de Rotações

O suporte para rotações contínuas foi implementado através da introdução do atributo `time` no elemento `rotate`. O ângulo de rotação deixa de ser fixo e passa a ser calculado dinamicamente:

$\theta = (T_{atual} \pmod{T_{ciclo}})/T_{ciclo} \times 360^\circ$

permitindo simular movimentos de rotação e translação orbital com precisão temporal.

4 Vertex Buffer Objects (VBOs)

Com o aumento da complexidade geométrica das cenas, nomeadamente com a introdução de superfícies de Bézier altamente tesseladas, a eficiência da renderização tornou-se uma prioridade fundamental. O modelo de programação anterior revelava-se inadequado para geometria de grande volume.

4.1 Transição do Paradigma de Desenho

No *Immediate Mode*, cada vértice era transmitido individualmente da memória do CPU para o GPU em cada *frame*, através de chamadas `glBegin`/`glVertex`/`glEnd`. Esta abordagem é ineficiente pois cada chamada implica uma transição do contexto do CPU para

o driver gráfico, os dados de geometria residem na RAM principal e têm de ser retransmitidos integralmente a cada ciclo de renderização e finalmente, o GPU não pode iniciar o processamento de um vértice antes de o CPU lhe enviar o seguinte.

Os VBOs resolvem estas limitações fundamentais. No novo fluxo de trabalho implementado:

1. **Inicialização**: Na função `loadModelVBO`, chamada uma única vez no arranque do motor, os dados dos modelos são lidos do disco e carregados para a VRAM através da seguinte sequência:
   - `glGenBuffers` gera um identificador único (`GLuint`) para cada buffer;
   - `glBindBuffer(GL_ARRAY_BUFFER, id)` seleciona o buffer como destino ativo;
   - `glBufferData(..., GL_STATIC_DRAW)` transfere o array de floats para a VRAM. A flag `GL_STATIC_DRAW` informa o driver de que a geometria não será modificada, permitindo-lhe otimizar o posicionamento físico do buffer em memória de acesso mais rápido.
2. **Ciclo de renderização**: Em cada *frame*, para cada modelo, o motor apenas executa: `glBindBuffer`, `glVertexPointer` e `glDrawArrays`. O GPU opera exclusivamente sobre os dados já residentes na VRAM.

4.2 Renderização de Alto Desempenho

A separação entre a fase de *upload* (feita uma vez) e a fase de *draw* (repetida a cada *frame*) é a principal vantagem dos VBOs. Durante o ciclo de renderização, a comunicação CPU–GPU resume-se à indicação de qual buffer usar e quantos vértices desenhar..

Esta abordagem é particularmente relevante para a cena do sistema solar: o modelo do teapot gerado com tesselação 10 contém aproximadamente 6 000 vértices, e a esfera de alta resolução usada nos planetas ultrapassa os 10 000 vértices. Com *Immediate Mode*, estes dados seriam retransmitidos integralmente em cada *frame*, mas com VBOs, residem permanentemente na VRAM e o GPU pode processá-los em paralelo sem intervenção do CPU.

5 Sistema Solar Dinâmico

Para validar a implementação da Fase 3, foi construído um sistema solar dinâmico que combina todas as funcionalidades desenvolvidas.

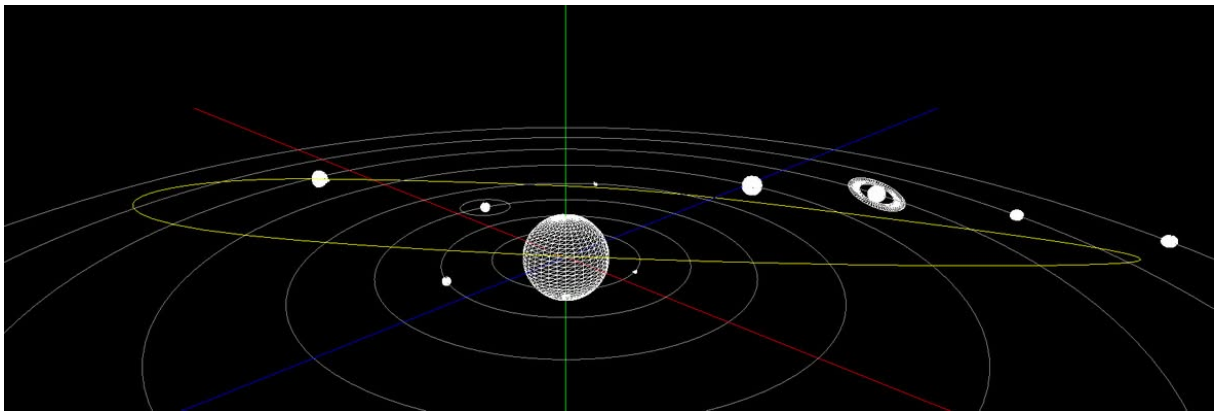
5.1 Hierarquia e Movimentos Orbitais

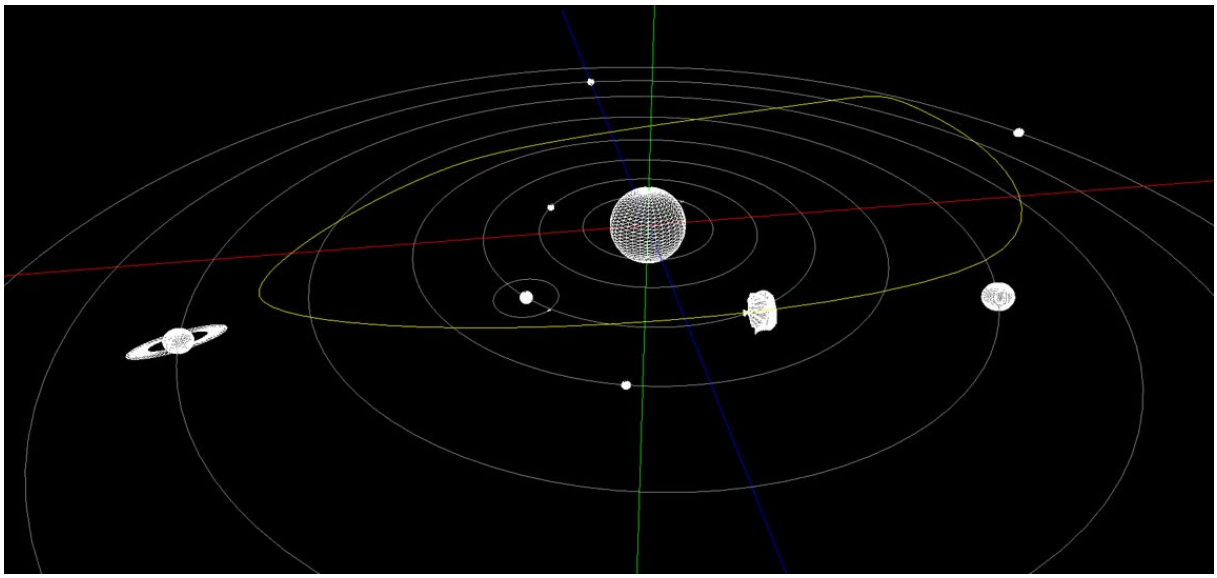
A cena utiliza a *Scene Graph* para modelar as relações entre corpos celestes. Cada planeta possui um movimento de rotação própria e uma translação orbital, ambos controlados por tempos definidos no XML, garantindo que as órbitas são permanentes e proporcionais.

Para auxiliar a compreensão espacial da cena, foi implementada uma função auxiliar (`renderOrbit`) que desenha as trajetórias circulares dos planetas. Esta função é invocada condicionalmente durante a travessia da *Scene Graph*: sempre que é encontrado um `rotate` com `time` seguido imediatamente de um `translate` estático (sem `time`), o motor interpreta esse padrão como uma órbita circular e desenha-a com o raio correspondente à distância de translação. Esta lógica não se aplica ao cometa, cuja trajetória é uma curva de Catmull-Rom e é visualizada pela função `renderCatmullRomCurve`.

5.2 O Cometa e as Superfícies de Bézier

O elemento central da demonstração é o cometa, cujo modelo (o *teapot*) é gerado dinamicamente a partir de *patches* de Bézier. O cometa percorre uma trajetória não circular definida por uma curva de Catmull-Rom que atravessa o plano da eclíptica. Através da funcionalidade de alinhamento, o modelo orienta-se automaticamente ao longo do percurso, demonstrando a correta integração entre a geometria paramétrica e as animações temporais.





Sistema Solar implementado

6 Conclusão

A terceira fase do projeto consolidou e expandiu significativamente as capacidades do motor gráfico desenvolvido nas fases anteriores, introduzindo três contribuições principais.

A geração de superfícies de Bézier no *generator* permite agora criar geometria paramétrica complexa a partir de ficheiros de *patches*, com controlo preciso do nível de detalhe através do parâmetro de tesselação. A implementação das curvas de Catmull-Rom no motor tornou possível animar objetos ao longo de trajetórias curvilíneas suaves e cíclicas, com suporte opcional para alinhamento automático à direção do movimento. A transição para VBOs modernizou a *pipeline* de renderização, eliminando o gargalo de comunicação CPU–GPU característico do *Immediate Mode* e tornando o motor apto a lidar com geometria de maior dimensão sem degradação de desempenho.

A cena de demonstração (sistema solar dinâmico) valida a integração de todas estas funcionalidades, pois temos planetas com órbitas e rotações próprias controladas por tempo, luas em hierarquia relativa ao planeta-pai, e um cometa cujo modelo (o *teapot* gerado por patches de Bézier) percorre uma trajetória irregular de Catmull-Rom orientando-se automaticamente ao longo do percurso.